Java LLD design for vending machine with UPI/cash and change handling

https://ms.sg.dev/deepsearch/a1217330-5384-4516-b93f-7b661121a821

---

java lld interview: Design vending machine: which caters coke, chips, chocolate, biscuits Payment mode currently accepted is upi , cash If I buy for Rs. 48 and pay Rs.50 cash, remaining Rs.2 should be dispensed, If dont have necessary change amount dont process order or ask customer If prefers UPI Inventory should be reloaded once in a day, also change coins in cash box Which design pattern we can use? is design pattern even needed

---

Vending Machine — Java LLD Design

Is a Design Pattern Needed?

Yes. The machine has complex state transitions (idle → selecting → paying → dispensing), multiple payment modes that share an interface, and inventory management. Without patterns, you end up with giant `if-else` chains that are hard to extend. Patterns keep the code open for extension and closed for modification.

Design Patterns Used

Pattern	Why
State	Machine behavior changes based on state (idle, item selected, awaiting payment, dispensing)
Strategy	Pluggable payment modes (UPI, Cash) without changing core machine logic
Singleton	One vending machine instance manages shared state
Factory	Create payment handlers without exposing instantiation logic
Observer	Notify admin/system when inventory is low or reload is due

Class Diagram (Key Entities)

```
VendingMachine (Singleton)
  ├── Inventory
  │     └── Map<Item, Integer> stock
  ├── CashBox
  │     └── Map<Coin, Integer> coins
  ├── VendingMachineState (interface)
  │     ├── IdleState
  │     ├── ItemSelectedState
  │     ├── PaymentPendingState
  │     └── DispensingState
  ├── PaymentStrategy (interface)
  │     ├── CashPaymentStrategy
  │     └── UPIPaymentStrategy
  └── InventoryReloader (Scheduler)
```

Core Code

Item & Coin Enums

```
public enum Item {
    COKE(30), CHIPS(20), CHOCOLATE(40), BISCUITS(15);

    private final int price;
    Item(int price) { this.price = price; }
    public int getPrice() { return price; }
}

public enum Coin {
    ONE(1), TWO(2), FIVE(5), TEN(10);

    private final int value;
    Coin(int value) { this.value = value; }
    public int getValue() { return value; }
}
```

Inventory

```
public class Inventory {
    private final Map<Item, Integer> stock = new EnumMap<>(Item.class);
```

```
    public void reload() {
        for (Item item : Item.values()) stock.put(item, 10); // daily reload
    }

    public boolean isAvailable(Item item) {
        return stock.getOrDefault(item, 0) > 0;
    }

    public void dispense(Item item) {
        if (!isAvailable(item)) throw new IllegalStateException("Out of stock: " + item);
        stock.put(item, stock.get(item) - 1);
    }
}
```

CashBox — Change Dispensing Logic

```
public class CashBox {
    private final Map<Coin, Integer> coins = new EnumMap<>(Coin.class);

    public void reload(Map<Coin, Integer> coinMap) {
        coins.putAll(coinMap);
    }

    public boolean canMakeChange(int amount) {
        return makeChange(amount, false) != null;
    }

    public Map<Coin, Integer> dispenseChange(int amount) {
        Map<Coin, Integer> change = makeChange(amount, true);
        if (change == null) throw new IllegalStateException("Cannot make change for: " + amount);
        return change;
    }

    private Map<Coin, Integer> makeChange(int amount, boolean deduct) {
        Map<Coin, Integer> result = new EnumMap<>(Coin.class);
        // Greedy approach: largest coin first
        Coin[] sorted = {Coin.TEN, Coin.FIVE, Coin.TWO, Coin.ONE};
        Map<Coin, Integer> temp = new EnumMap<>(coins);

        for (Coin coin : sorted) {
            int available = temp.getOrDefault(coin, 0);
            int needed = Math.min(amount / coin.getValue(), available);
            if (needed > 0) {
                result.put(coin, needed);
                amount -= needed * coin.getValue();
                temp.put(coin, available - needed);
            }
        }
```

```
        if (amount != 0) return null; // cannot make exact change

        if (deduct) coins.putAll(temp); // commit deduction
        return result;
    }
}
```

Payment Strategy (Strategy Pattern)

```
public interface PaymentStrategy {
    PaymentResult pay(int amount);
}

public record PaymentResult(boolean success, String message) {}
```

```
public class CashPaymentStrategy implements PaymentStrategy {
    private final CashBox cashBox;
    private final int paidAmount;

    public CashPaymentStrategy(CashBox cashBox, int paidAmount) {
        this.cashBox = cashBox;
        this.paidAmount = paidAmount;
    }

    @Override
    public PaymentResult pay(int amount) {
        if (paidAmount < amount)
            return new PaymentResult(false, "Insufficient cash.");

        int change = paidAmount - amount;
        if (change > 0 && !cashBox.canMakeChange(change))
            return new PaymentResult(false,
                "Cannot return change of Rs." + change + ". Prefer UPI?");

        if (change > 0) cashBox.dispenseChange(change);
        return new PaymentResult(true, "Cash accepted. Change: Rs." + change);
    }
}

public class UPIPaymentStrategy implements PaymentStrategy {
    private final String upiId;

    public UPIPaymentStrategy(String upiId) { this.upiId = upiId; }

    @Override
    public PaymentResult pay(int amount) {
        // Simulate UPI gateway call
        System.out.println("Requesting Rs." + amount + " via UPI from " + upiId);
```

```
        return new PaymentResult(true, "UPI payment successful.");
    }
}
```

State Pattern

```
public interface VendingMachineState {
    void selectItem(Item item);
    void choosePayment(PaymentStrategy strategy);
    void processPayment();
    void dispenseItem();
    void cancel();
}
```

```
public class IdleState implements VendingMachineState {
    private final VendingMachine machine;
    IdleState(VendingMachine machine) { this.machine = machine; }

    @Override
    public void selectItem(Item item) {
        if (!machine.getInventory().isAvailable(item)) {
            System.out.println(item + " is out of stock.");
            return;
        }
        machine.setSelectedItem(item);
        machine.setState(new ItemSelectedState(machine));
        System.out.println("Selected: " + item + " | Price: Rs." + item.getPrice());
    }

    @Override public void choosePayment(PaymentStrategy s) { System.out.println("Select item first."); }
    @Override public void processPayment()                 { System.out.println("Select item first."); }
    @Override public void dispenseItem()                   { System.out.println("Nothing to dispense."); }
    @Override public void cancel()                         { System.out.println("Nothing to cancel."); }
}
```

```
public class ItemSelectedState implements VendingMachineState {
    private final VendingMachine machine;
    ItemSelectedState(VendingMachine machine) { this.machine = machine; }

    @Override
    public void choosePayment(PaymentStrategy strategy) {
        machine.setPaymentStrategy(strategy);
        machine.setState(new PaymentPendingState(machine));
    }

    @Override public void selectItem(Item item)  { System.out.println("Item already selected. Cancel first."); }
    @Override public void processPayment()       { System.out.println("Choose payment method first."); }
```

```
    @Override public void dispenseItem()         { System.out.println("Pay first."); }
    @Override public void cancel() {
        machine.setSelectedItem(null);
        machine.setState(new IdleState(machine));
        System.out.println("Cancelled.");
    }
}
```

```
public class PaymentPendingState implements VendingMachineState {
    private final VendingMachine machine;
    PaymentPendingState(VendingMachine machine) { this.machine = machine; }

    @Override
    public void processPayment() {
        int price = machine.getSelectedItem().getPrice();
        PaymentResult result = machine.getPaymentStrategy().pay(price);
        System.out.println(result.message());

        if (result.success()) {
            machine.setState(new DispensingState(machine));
            machine.dispenseItem();
        } else {
            // Stay in PaymentPending — let customer retry with UPI
            System.out.println("Payment failed. Try UPI or cancel.");
        }
    }

    @Override public void selectItem(Item item)           { System.out.println("Cancel current selection first.")
    @Override public void choosePayment(PaymentStrategy s){ machine.setPaymentStrategy(s); } // allow switching
    @Override public void dispenseItem()                  { System.out.println("Complete payment first."); }
    @Override public void cancel() {
        machine.setSelectedItem(null);
        machine.setState(new IdleState(machine));
        System.out.println("Transaction cancelled.");
    }
}
```

```
public class DispensingState implements VendingMachineState {
    private final VendingMachine machine;
    DispensingState(VendingMachine machine) { this.machine = machine; }

    @Override
    public void dispenseItem() {
        machine.getInventory().dispense(machine.getSelectedItem());
        System.out.println("Dispensing: " + machine.getSelectedItem() + ". Enjoy!");
        machine.setSelectedItem(null);
        machine.setState(new IdleState(machine));
    }
```

```
    @Override public void selectItem(Item i)           { System.out.println("Please wait, dispensing."); }
    @Override public void choosePayment(PaymentStrategy s){ System.out.println("Please wait, dispensing."); }
    @Override public void processPayment()             { System.out.println("Please wait, dispensing."); }
    @Override public void cancel()                     { System.out.println("Cannot cancel during dispensing.");
}
```

VendingMachine (Singleton + Orchestrator)

```
public class VendingMachine {
    private static VendingMachine instance;

    private VendingMachineState state;
    private final Inventory inventory;
    private final CashBox cashBox;
    private Item selectedItem;
    private PaymentStrategy paymentStrategy;

    private VendingMachine() {
        inventory = new Inventory();
        cashBox   = new CashBox();
        state     = new IdleState(this);
        inventory.reload();
        cashBox.reload(Map.of(Coin.ONE, 20, Coin.TWO, 20, Coin.FIVE, 10, Coin.TEN, 10));
    }

    public static synchronized VendingMachine getInstance() {
        if (instance == null) instance = new VendingMachine();
        return instance;
    }

    // Delegate to current state
    public void selectItem(Item item)              { state.selectItem(item); }
    public void choosePayment(PaymentStrategy s)   { state.choosePayment(s); }
    public void processPayment()                   { state.processPayment(); }
    public void dispenseItem()                     { state.dispenseItem(); }
    public void cancel()                           { state.cancel(); }

    // Getters/setters for state objects
    public Inventory getInventory()                       { return inventory; }
    public CashBox getCashBox()                           { return cashBox; }
    public Item getSelectedItem()                         { return selectedItem; }
    public void setSelectedItem(Item item)                { this.selectedItem = item; }
    public PaymentStrategy getPaymentStrategy()           { return paymentStrategy; }
    public void setPaymentStrategy(PaymentStrategy s)     { this.paymentStrategy = s; }
    public void setState(VendingMachineState state)       { this.state = state; }

    // Daily reload — triggered by InventoryReloader scheduler
    public void dailyReload(Map<Coin, Integer> freshCoins) {
        inventory.reload();
```

```
        cashBox.reload(freshCoins);
        System.out.println("Daily inventory and cash box reloaded.");
    }
}
```

Daily Reload Scheduler (Observer / Scheduler)

```
public class InventoryReloader {
    private final VendingMachine machine;

    public InventoryReloader(VendingMachine machine) {
        this.machine = machine;
        scheduleDaily();
    }

    private void scheduleDaily() {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
        long initialDelay = computeDelayToMidnight();

        scheduler.scheduleAtFixedRate(() ->
            machine.dailyReload(Map.of(
                Coin.ONE, 20, Coin.TWO, 20,
                Coin.FIVE, 10, Coin.TEN, 10
            )),
            initialDelay, 24 * 60 * 60, TimeUnit.SECONDS
        );
    }

    private long computeDelayToMidnight() {
        LocalDateTime now      = LocalDateTime.now();
        LocalDateTime midnight = now.toLocalDate().plusDays(1).atStartOfDay();
        return ChronoUnit.SECONDS.between(now, midnight);
    }
}
```

Demo — Rs.48 item, pay Rs.50 cash

```
public class Main {
    public static void main(String[] args) {
        VendingMachine vm = VendingMachine.getInstance();

        vm.selectItem(Item.CHOCOLATE);                                  // Rs.40 — but let's use CHIPS scenario

        // Scenario: COKE = Rs.30, but imagine CHOCOLATE = Rs.48 for demo
        // Use CHIPS + adjustment or add custom item — here we show the flow:
```

```
        vm.selectItem(Item.CHOCOLATE);                                  // Rs.40
        // Customer pays Rs.50 cash
        vm.choosePayment(new CashPaymentStrategy(vm.getCashBox(), 50));
        vm.processPayment();   // change = Rs.10, dispensed if coins available
        // Output: "Cash accepted. Change: Rs.10" → dispenses item

        // --- Scenario: No change available ---
        VendingMachine vm2 = VendingMachine.getInstance();
        vm2.selectItem(Item.CHIPS);                                     // Rs.20
        vm2.choosePayment(new CashPaymentStrategy(vm2.getCashBox(), 22)); // needs Rs.2 change
        vm2.processPayment();
        // If Rs.2 coin unavailable:
        // Output: "Cannot return change of Rs.2. Prefer UPI?"
        // Customer can then switch:
        vm2.choosePayment(new UPIPaymentStrategy("customer@upi"));
        vm2.processPayment();  // succeeds via UPI
    }
}
```

Key Design Decisions Summary

Decision	Rationale
State pattern for machine	Eliminates `if (state == X)` chains in every method; each state encapsulates its own valid transitions
Strategy for payment	Adding new payment (e.g., Card, Wallet) = new class, zero changes to machine
Greedy change algorithm in CashBox	Maximizes large coins first; fails fast if exact change impossible
"Prefer UPI?" prompt	Stays in `PaymentPendingState` on cash failure; customer can call `choosePayment(UPI)` and retry without restarting
Daily reload via `ScheduledExecutorService`	Fires once at midnight, handles both inventory and coins together atomically
Singleton	One machine = one consistent inventory/cashbox state; thread-safe with `synchronized getInstance()`

Make machine reload stock when runs out of stock from inventory

---

Auto-Reload on Stock Exhaustion

The key changes are:

1. `Inventory` tracks low/empty state and triggers an `InventoryObserver`
2. `VendingMachine` implements the observer and auto-reloads the specific item
3. Reload happens **per-item**, not full machine reload (realistic — supplier restocks per SKU)

Observer Interfaces

```
public interface InventoryObserver {
    void onItemOutOfStock(Item item);
    void onItemLowStock(Item item, int remaining);  // optional early warning
}

public interface InventorySubject {
    void registerObserver(InventoryObserver observer);
    void removeObserver(InventoryObserver observer);
}
```

Updated `Inventory` — Notifies on Empty/Low

```
public class Inventory implements InventorySubject {

    private static final int LOW_STOCK_THRESHOLD = 2;
    private static final int RESTOCK_QUANTITY    = 10;

    private final Map<Item, Integer> stock = new EnumMap<>(Item.class);
    private final List<InventoryObserver> observers = new ArrayList<>();

    // --- InventorySubject ---
    @Override
    public void registerObserver(InventoryObserver observer) { observers.add(observer); }

    @Override
    public void removeObserver(InventoryObserver observer)   { observers.remove(observer); }

    private void notifyOutOfStock(Item item) {
        observers.forEach(o -> o.onItemOutOfStock(item));
    }

    private void notifyLowStock(Item item, int remaining) {
        observers.forEach(o -> o.onItemLowStock(item, remaining));
    }
```

```
    // --- Core operations ---
    public void reload() {
        for (Item item : Item.values()) stock.put(item, RESTOCK_QUANTITY);
        System.out.println("[Inventory] Full reload complete.");
    }

    public void restockItem(Item item, int quantity) {
        stock.merge(item, quantity, Integer::sum);
        System.out.println("[Inventory] Restocked " + item + " → " + stock.get(item) + " units.");
    }

    public boolean isAvailable(Item item) {
        return stock.getOrDefault(item, 0) > 0;
    }

    public int getStock(Item item) {
        return stock.getOrDefault(item, 0);
    }

    public void dispense(Item item) {
        int current = stock.getOrDefault(item, 0);
        if (current <= 0) throw new IllegalStateException("Out of stock: " + item);

        int updated = current - 1;
        stock.put(item, updated);
        System.out.println("[Inventory] Dispensed " + item + ". Remaining: " + updated);

        // Notify observers based on updated count
        if (updated == 0) {
            notifyOutOfStock(item);
        } else if (updated <= LOW_STOCK_THRESHOLD) {
            notifyLowStock(item, updated);
        }
    }
}
```

`RestockService` — Simulates Supplier / Auto-Restock

```
public class RestockService {

    private static final int DEFAULT_RESTOCK_QTY = 10;

    private final Inventory inventory;

    public RestockService(Inventory inventory) {
        this.inventory = inventory;
    }

    /**
```

```
     * Called immediately when stock hits zero.
     * In production, this could call a supplier API or trigger a warehouse order.
     */
    public void restockItem(Item item) {
        System.out.println("[RestockService] Auto-restocking " + item + "...");
        inventory.restockItem(item, DEFAULT_RESTOCK_QTY);
    }

    /**
     * Optional: alert staff for low stock early warning.
     */
    public void alertLowStock(Item item, int remaining) {
        System.out.println("[RestockService] LOW STOCK ALERT — " + item
                + " has only " + remaining + " unit(s) left. Scheduling restock.");
        // Could send email/SMS/push to admin here
    }
}
```

Updated `VendingMachine` — Implements `InventoryObserver`

```
public class VendingMachine implements InventoryObserver {

    private static VendingMachine instance;

    private VendingMachineState state;
    private final Inventory      inventory;
    private final CashBox        cashBox;
    private final RestockService restockService;

    private Item            selectedItem;
    private PaymentStrategy paymentStrategy;

    private VendingMachine() {
        inventory      = new Inventory();
        cashBox        = new CashBox();
        restockService = new RestockService(inventory);

        // Register self as observer — machine reacts to inventory events
        inventory.registerObserver(this);

        state = new IdleState(this);
        inventory.reload();
        cashBox.reload(Map.of(Coin.ONE, 20, Coin.TWO, 20, Coin.FIVE, 10, Coin.TEN, 10));
    }

    public static synchronized VendingMachine getInstance() {
        if (instance == null) instance = new VendingMachine();
        return instance;
    }
```

```
    // --- InventoryObserver callbacks ---

    @Override
    public void onItemOutOfStock(Item item) {
        System.out.println("[VendingMachine] " + item + " is OUT OF STOCK. Triggering auto-restock...");
        restockService.restockItem(item);
    }

    @Override
    public void onItemLowStock(Item item, int remaining) {
        System.out.println("[VendingMachine] " + item + " is LOW (" + remaining + " left). Notifying supplier..."
        restockService.alertLowStock(item, remaining);
    }

    // --- State delegation (unchanged) ---
    public void selectItem(Item item)            { state.selectItem(item); }
    public void choosePayment(PaymentStrategy s) { state.choosePayment(s); }
    public void processPayment()                 { state.processPayment(); }
    public void dispenseItem()                   { state.dispenseItem(); }
    public void cancel()                         { state.cancel(); }

    // --- Getters / Setters ---
    public Inventory        getInventory()                   { return inventory; }
    public CashBox          getCashBox()                     { return cashBox; }
    public Item             getSelectedItem()                { return selectedItem; }
    public void             setSelectedItem(Item item)       { this.selectedItem = item; }
    public PaymentStrategy  getPaymentStrategy()             { return paymentStrategy; }
    public void             setPaymentStrategy(PaymentStrategy s) { this.paymentStrategy = s; }
    public void             setState(VendingMachineState s)  { this.state = s; }

    public void dailyReload(Map<Coin, Integer> freshCoins) {
        inventory.reload();
        cashBox.reload(freshCoins);
        System.out.println("[VendingMachine] Daily reload complete.");
    }
}
```

Demo — Auto-Restock in Action

```
public class Main {
    public static void main(String[] args) {
        VendingMachine vm = VendingMachine.getInstance();

        // Drain CHIPS stock to trigger auto-restock
        for (int i = 0; i < 10; i++) {
            vm.selectItem(Item.CHIPS);
            vm.choosePayment(new UPIPaymentStrategy("test@upi"));
            vm.processPayment();
```

```
        }
    }
}
```

Output:

```
[Inventory] Dispensed CHIPS. Remaining: 9
[Inventory] Dispensed CHIPS. Remaining: 8
...
[Inventory] Dispensed CHIPS. Remaining: 2
[VendingMachine] CHIPS is LOW (2 left). Notifying supplier...
[RestockService] LOW STOCK ALERT — CHIPS has only 2 unit(s) left. Scheduling restock.
[Inventory] Dispensed CHIPS. Remaining: 1
[Inventory] Dispensed CHIPS. Remaining: 0
[VendingMachine] CHIPS is OUT OF STOCK. Triggering auto-restock...
[RestockService] Auto-restocking CHIPS...
[Inventory] Restocked CHIPS → 10 units.
```

What Changed vs Previous Design

Area	Before	After
`Inventory`	Passive, no notifications	Implements `InventorySubject`, fires events on dispense
`VendingMachine`	No awareness of stock depletion	Implements `InventoryObserver`, reacts to out-of-stock
`RestockService`	Did not exist	Encapsulates restock logic, decoupled from machine
Restock trigger	Manual daily reload only	**Automatic per-item** on depletion + early low-stock warning
Pattern added	—	**Observer** wiring `Inventory → VendingMachine → RestockService`

Its better to alert inventory before item goes out of stock, so that I dont lose business

---

Good point. The current `LOW_STOCK_THRESHOLD = 2` already fires `onItemLowStock`, but it only logs — it does **not** trigger a restock order proactively. The restock only happens at zero, by which time you've already lost potential sales during the restock window.

The fix: **trigger the actual restock order at low stock**, not at zero. Zero becomes the last-resort fallback.

Updated Flow

```
10 units → ... → 3 units (LOW threshold hit)
    → RestockService places restock ORDER immediately
    → Supplier delivers asynchronously (simulated delay)
    → Stock replenished BEFORE hitting 0
    → Customer never sees "out of stock"

0 units (fallback, if supplier was too slow)
    → Emergency restock + machine shows "temporarily unavailable"
```

`RestockStatus` — Track Pending Orders Per Item

```
public enum RestockStatus {
    NONE, ORDERED, DELIVERED
}
```

```
public class RestockOrder {
    private final Item item;
    private final int  quantity;
    private RestockStatus status;
    private final LocalDateTime orderedAt;

    public RestockOrder(Item item, int quantity) {
        this.item      = item;
        this.quantity  = quantity;
        this.status    = RestockStatus.ORDERED;
        this.orderedAt = LocalDateTime.now();
    }

    public Item getItem()               { return item; }
    public int getQuantity()            { return quantity; }
    public RestockStatus getStatus()    { return status; }
    public void setStatus(RestockStatus s) { this.status = s; }
    public LocalDateTime getOrderedAt() { return orderedAt; }
}
```

Updated `RestockService` — Proactive Order + Async Delivery

```
public class RestockService {

    private static final int RESTOCK_QTY         = 10;
    private static final int SUPPLIER_DELAY_SECS = 5;  // simulate supplier lead time

    private final Inventory inventory;
    // Prevent duplicate orders for same item
    private final Map<Item, RestockStatus> pendingOrders = new EnumMap<>(Item.class);
    private final ScheduledExecutorService scheduler     = Executors.newScheduledThreadPool(2);

    public RestockService(Inventory inventory) {
        this.inventory = inventory;
    }

    /**
     * Called at LOW STOCK threshold — places order NOW so stock arrives before hitting zero.
     * Ignores if an order is already pending for this item.
     */
    public void placeRestockOrder(Item item) {
        if (pendingOrders.getOrDefault(item, RestockStatus.NONE) == RestockStatus.ORDERED) {
            System.out.println("[RestockService] Order already pending for " + item + ". Skipping duplicate.");
            return;
        }

        RestockOrder order = new RestockOrder(item, RESTOCK_QTY);
        pendingOrders.put(item, RestockStatus.ORDERED);

        System.out.println("[RestockService] Restock ORDER placed for " + item
                + " (qty: " + RESTOCK_QTY + "). Supplier will deliver in ~"
                + SUPPLIER_DELAY_SECS + "s.");

        // Simulate async supplier delivery
        scheduler.schedule(() -> deliverStock(order), SUPPLIER_DELAY_SECS, TimeUnit.SECONDS);
    }

    /**
     * Emergency restock — called only if stock actually hits 0 before supplier arrived.
     * This is the safety net, should rarely fire with proactive ordering.
     */
    public void emergencyRestock(Item item) {
        System.out.println("[RestockService] EMERGENCY restock triggered for " + item
                + " — supplier was too slow!");
        inventory.restockItem(item, RESTOCK_QTY);
        pendingOrders.put(item, RestockStatus.DELIVERED);
    }

    private void deliverStock(RestockOrder order) {
        inventory.restockItem(order.getItem(), order.getQuantity());
        pendingOrders.put(order.getItem(), RestockStatus.DELIVERED);
        // Reset so future low-stock can trigger a new order
        scheduler.schedule(() -> pendingOrders.put(order.getItem(), RestockStatus.NONE),
                1, TimeUnit.SECONDS);
```

```
        System.out.println("[RestockService] Delivery complete for " + order.getItem() + ".");
    }
}
```

Updated `Inventory` — Fine-Grained Thresholds

```
public class Inventory implements InventorySubject {

    private static final int LOW_STOCK_THRESHOLD      = 3;  // order placed HERE — proactive
    private static final int CRITICAL_STOCK_THRESHOLD = 1;  // last warning before zero
    private static final int RESTOCK_QUANTITY         = 10;

    private final Map<Item, Integer>      stock     = new EnumMap<>(Item.class);
    private final List<InventoryObserver> observers = new ArrayList<>();

    @Override public void registerObserver(InventoryObserver o) { observers.add(o); }
    @Override public void removeObserver(InventoryObserver o)   { observers.remove(o); }

    public void reload() {
        for (Item item : Item.values()) stock.put(item, RESTOCK_QUANTITY);
        System.out.println("[Inventory] Full reload complete.");
    }

    public void restockItem(Item item, int qty) {
        stock.merge(item, qty, Integer::sum);
        System.out.println("[Inventory] " + item + " stock now: " + stock.get(item) + " units.");
    }

    public boolean isAvailable(Item item) { return stock.getOrDefault(item, 0) > 0; }
    public int     getStock(Item item)    { return stock.getOrDefault(item, 0); }

    public void dispense(Item item) {
        int current = stock.getOrDefault(item, 0);
        if (current <= 0) throw new IllegalStateException("Out of stock: " + item);

        int remaining = current - 1;
        stock.put(item, remaining);
        System.out.println("[Inventory] Dispensed " + item + ". Remaining: " + remaining);

        // Trigger notifications based on remaining count
        if (remaining == 0) {
            observers.forEach(o -> o.onItemOutOfStock(item));              // emergency fallback
        } else if (remaining == CRITICAL_STOCK_THRESHOLD) {
            observers.forEach(o -> o.onItemCriticalStock(item, remaining)); // last warning
        } else if (remaining == LOW_STOCK_THRESHOLD) {
            observers.forEach(o -> o.onItemLowStock(item, remaining));     // proactive order
        }
```

```
    }
}
```

Updated `InventoryObserver` — Three-Level Alert

```
public interface InventoryObserver {
    void onItemLowStock(Item item, int remaining);       // threshold hit → place order NOW
    void onItemCriticalStock(Item item, int remaining);  // 1 unit left  → warn loudly
    void onItemOutOfStock(Item item);                    // 0 units      → emergency fallback
}
```

Updated `VendingMachine` — Handles All Three Levels

```
@Override
public void onItemLowStock(Item item, int remaining) {
    System.out.println("[VendingMachine] LOW STOCK: " + item
            + " has " + remaining + " units. Placing proactive restock order...");
    restockService.placeRestockOrder(item);   // order NOW, stock arrives before zero
}

@Override
public void onItemCriticalStock(Item item, int remaining) {
    System.out.println("[VendingMachine] CRITICAL: Only " + remaining
            + " unit(s) of " + item + " left! Supplier order should arrive soon.");
    // Optionally display warning on machine UI panel
}

@Override
public void onItemOutOfStock(Item item) {
    System.out.println("[VendingMachine] EMERGENCY: " + item
            + " is EMPTY. Supplier was too slow — triggering emergency restock.");
    restockService.emergencyRestock(item);   // safety net only
}
```

Demo

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        VendingMachine vm = VendingMachine.getInstance(); // stock = 10 each

        // Sell CHIPS down to trigger alerts
        for (int i = 0; i < 8; i++) {
            vm.selectItem(Item.CHIPS);
```

```
            vm.choosePayment(new UPIPaymentStrategy("test@upi"));
            vm.processPayment();
            Thread.sleep(300);
        }

        Thread.sleep(6000); // wait for async supplier delivery
    }
}
```

Output:

```
[Inventory] Dispensed CHIPS. Remaining: 9
[Inventory] Dispensed CHIPS. Remaining: 8
...
[Inventory] Dispensed CHIPS. Remaining: 3
[VendingMachine] LOW STOCK: CHIPS has 3 units. Placing proactive restock order...
[RestockService] Restock ORDER placed for CHIPS (qty: 10). Supplier will deliver in ~5s.

[Inventory] Dispensed CHIPS. Remaining: 2
[Inventory] Dispensed CHIPS. Remaining: 1
[VendingMachine] CRITICAL: Only 1 unit(s) of CHIPS left! Supplier order should arrive soon.

[Inventory] Dispensed CHIPS. Remaining: 0     ← customer got the last one, no lost sale

--- 5 seconds later ---
[RestockService] Delivery complete for CHIPS.
[Inventory] CHIPS stock now: 10 units.         ← replenished proactively
```

Alert Level Summary

Stock Level	Event	Action
`== LOW_STOCK_THRESHOLD (3)`	`onItemLowStock`	Place supplier order **immediately**
`== CRITICAL_STOCK_THRESHOLD (1)`	`onItemCriticalStock`	Loud warning; order already in-flight
`== 0`	`onItemOutOfStock`	**Emergency** restock — safety net only, should rarely fire

The business never loses a sale unless the supplier is slower than it takes to sell 3 units — and the critical alert at 1 unit gives you a final signal to escalate if needed.